Java 基础与集合面试八股笔记

面向 Go 后端 / AI Agent 开发实习

Java 基础与集合面试八股笔记

一、Java 概述

1. Java 特点与跨平台原理
2. JDK、JRE、JVM
3. 编译与解释并存
4. 值传递

二、数据类型

5. 八种基本数据类型
6. 类型转换与常见问题
7. 包装类、装箱拆箱与 Integer 缓存
8. BigDecimal 与浮点精度

三、面向对象

9. 封装、继承、多态
10. 重载与重写
11. 抽象类与接口
12. 面向对象设计原则（SOLID + 迪米特）
13. 内部类、static 与 final
14. 深拷贝与浅拷贝、对象创建方式

四、常用类与关键方法

15. String 不可变与字符串常量池
16. String、StringBuilder、StringBuffer
17. == 与 equals、hashCode
18. Object 类的方法

五、泛型、反射与注解

19. 泛型与类型擦除
20. 反射
21. 注解

六、异常

22. 异常体系
23. 异常处理机制

七、Java 新特性、序列化与 IO

24. Java 8 新特性
25. Java 17 / 21 常见新特性
26. 序列化

27. BIO、NIO、AIO
八、集合概览
28. 集合体系
29. 线程安全的集合
30. fail-fast 与 fail-safe
九、List
31. ArrayList 与 LinkedList
32. ArrayList 扩容机制
33. ArrayList 线程不安全的表现与解决
34. List 使用细节
十、Set
35. HashSet 去重原理
36. LinkedHashSet、TreeSet 与 Set 排序
十一、Map
37. HashMap 底层结构（JDK 8）
38. hash 计算与容量为 2 的幂
39. put 与 get 流程
40. 扩容机制
41. 为什么用红黑树而不是 AVL / 二叉搜索树
42. HashMap 线程安全问题
43. 哈希冲突解决方法与 key 的选择
44. Hashtable、LinkedHashMap、TreeMap
45. ConcurrentHashMap 实现原理
46. ConcurrentHashMap 扩容、size 与 null 限制
47. HashMap、Hashtable、ConcurrentHashMap 对比
十二、Queue
48. Queue、Deque 与 PriorityQueue
49. BlockingQueue

> 使用说明：按「主题 → 知识点」组织的结构化笔记，知识框架主要参考小林coding 面试题合集（Java 基础、Java 集合），已重新整理改写。只保留**基础与集合**（不含 JVM、JUC 并发、Spring）。先记加粗关键词，再展开。涉及版本差异处已标注版本号。Go 见 Go 面试八股笔记。

Java 基础与集合面试八股笔记

一、Java 概述

1. Java 特点与跨平台原理

- 特点：**平台无关**（一次编译，到处运行）、**面向对象**、**自动内存管理（GC）**、生态成熟（Spring 等）、内置多线程支持。
- 劣势：相对 C++/Rust/Go 启动慢、内存占用大，语法样板代码较多。
- **跨平台原理**：`javac` 把源码编译为平台无关的**字节码（.class）**，由各平台对应的 **JVM** 翻译成本地机器码执行。**跨平台的是字节码，JVM 本身不跨平台**（不同系统安装不同 JVM）。
- JVM 上不止能跑 Java，Kotlin、Scala 等编译为字节码同样可运行。

2. JDK、JRE、JVM

- **JVM**：运行字节码的虚拟机，负责解释/编译执行、内存管理、GC。
- **JRE** = JVM + 核心类库，运行 Java 程序的最小环境。
- **JDK** = JRE + 开发工具（`javac`、`jdb`、`jar`、`javap` 等）。关系：**JDK ⊃ JRE ⊃ JVM**（JDK 11 起官方不再单独提供 JRE 安装包）。

3. 编译与解释并存

- 编译期：`javac` 生成字节码。
- 运行期：**解释器**逐条解释执行；JVM 用方法调用计数器、回边计数器识别**热点代码**，由 **JIT 即时编译器**编译为本地机器码缓存，后续直接执行。
- 因此 Java 是**编译 + 解释混合**模式；C/C++ 是典型编译型，Python/JS 是典型解释型。

4. 值传递

- **Java 只有值传递**。
- 基本类型：传递值的副本，方法内修改不影响原变量。
- 引用类型：传递**引用（地址）的副本**——通过副本修改对象内部字段会影响原对象；但让副本指向新对象，不影响原引用。

二、数据类型

5. 八种基本数据类型

类型	字节	范围 / 说明	默认值	包装类
byte	1	-128 ~ 127	0	Byte
short	2	-2^15 ~ 2^15-1	0	Short
int	4	-2^31 ~ 2^31-1	0	Integer
long	8	-2^63 ~ 2^63-1，字面量加 L	0L	Long
float	4	单精度，字面量加 f	0.0f	Float
double	8	双精度，小数默认类型	0.0d	Double
char	2	无符号，0 ~ 65535（UTF-16 码元）	'000'	Character
boolean	未明确规定	true / false	false	Boolean

- 整数字面量默认 int，小数默认 double。

6. 类型转换与常见问题

- **自动（隐式）转换**：小范围 → 大范围（int → long，float → double），安全。
- **强制转换**：大范围 → 小范围，可能**溢出**（如 `(byte)300 == 44`，保留低 8 位）或**精度丢失**（`(int)3.14 == 3`）。
- long ↔ int：int → long 安全；long → int 需强转，超出范围时截断高位。
- **引用类型转换**：向上转型自动且安全；向下转型需强转，实际类型不符抛 `ClassCastException`，先用 `instanceof` 判断（Java 16 起支持 `if (obj instanceof Dog d)` 模式匹配）。
- 字符串转换：`Integer.parseInt()`、`Double.parseDouble()`、`String.valueOf()`。

7. 包装类、装箱拆箱与 Integer 缓存

- **装箱** `Integer i = 10;` 编译为 `Integer.valueOf(10)`；**拆箱** `int n = i;` 编译为 `i.intValue()`。发生在赋值和方法调用时。
- **为什么需要 Integer**：泛型/集合只能存引用类型（`List<Integer>`）；可用 `null` 表示「无值」；提供 `parseInt`、`toHexString` 等工具方法。
- **为什么保留 int**：基本类型直接存值，读写效率高；64 位 JVM 开启压缩指针时一个 Integer 对象约 16 字节，int 仅 4 字节。
- **IntegerCache**：`valueOf` 缓存 **-128 ~ 127**（上限可通过 JVM 参数调大），范围内复用同一对象。
  - `Integer a=127, b=127; a==b` → true；`128 == 128`（两个 Integer）→ false（默认缓存范围下）。

- **Integer 比较数值用 `equals`**（兼容 null 用 `Objects.equals`）；Integer 与 int 用 `==` 比较会先拆箱。

- 坑：**null 拆箱抛 NPE**；循环中 `Integer sum += i` 会反复装箱创建大量对象；`new Integer()` 自 JDK 9 起废弃。

8. BigDecimal 与浮点精度

- double 是**二进制浮点数**，无法精确表示 0.1 等十进制小数：`0.1 + 0.2 = 0.30000000000000004`。
- **金额等精确计算用 BigDecimal**（任意精度整数 + scale）。
- 注意：
  - 用**字符串构造** `new BigDecimal("0.1")` 或 `BigDecimal.valueOf(0.1)`，不要 `new BigDecimal(0.1)`；
  - **除法要指定精度和舍入模式**（如 `RoundingMode.HALF_UP`），否则除不尽抛 `ArithmeticException`；
  - **比较数值用 `compareTo`**，`equals` 会同时比较 scale（`1.0` 与 `1.00` 不 equals）；
  - BigDecimal 不可变，运算结果要接住返回值。

三、面向对象

9. 封装、继承、多态

- **封装**：属性和行为结合，隐藏内部细节，通过公开接口交互（private 字段 + getter/setter）。
- **继承**：子类复用父类属性和方法，Java **类单继承、接口多实现**。
- **多态**：同一接口不同实现。
  - **编译时多态**：方法重载；
  - **运行时多态**：方法重写 + 父类引用指向子类对象，运行时按**实际类型**动态分派。
- 多态的意义：提高扩展性，是策略模式、依赖倒置、消除 if-else 等设计的基础。

10. 重载与重写

对比	重载 Overload	重写 Override
位置	同一类（或继承得到的方法）	子类与父类之间
方法签名	方法名同，**参数列表不同**	方法名、参数列表**相同**
返回值	无要求（**不能仅靠返回值区分**）	相同或为其子类型（协变返回）
访问权限	无要求	**不能比父类更严格**
异常	无要求	不能抛出新的或更宽的受检异常
确定时机	**编译期**	**运行期**

- `final` 方法不能重写；`private` 方法不被继承；`static` 方法同名属于**隐藏**而非重写。建议加 `@Override` 让编译器检查。

11. 抽象类与接口

- **抽象类**：`abstract` 修饰，不能实例化（可有构造器，供子类调用）；可有成员变量、构造方法、具体方法和抽象方法；单继承；不能用 `final` 修饰（互斥）。
- **接口**：定义行为规范，可多实现；成员变量默认 `public static final`；不能有构造方法。
  - 抽象方法默认 `public abstract`；
  - **Java 8**：`default` 默认方法、`static` 静态方法；
  - **Java 9**：`private` 私有方法（辅助 default 方法）。
- 选择：表达「**is-a**」、需要共享状态和代码 → 抽象类；表达「**能力 / can-do**」、需要多实现 → 接口。

12. 面向对象设计原则（SOLID + 迪米特）

- **单一职责 SRP**：一个类只负责一项职责。
- **开闭原则 OCP**：对扩展开放，对修改关闭。
- **里氏替换 LSP**：子类对象能替换父类对象而不改变程序正确性。
- **接口隔离 ISP**：接口小而专，客户端不依赖不需要的方法。
- **依赖倒置 DIP**：高层和低层都依赖抽象，面向接口编程。
- **迪米特法则**：只与直接朋友通信，减少耦合。

13. 内部类、static 与 final

- **非静态内部类**：持有外部类实例引用（编译器在构造方法中传入外部类对象），可直接访问外部类所有成员；**可能导致外部类无法回收（内存泄漏）**。
- **静态内部类**：不依赖外部类实例，只能直接访问外部类静态成员（单例的静态内部类写法即利用其延迟加载）。
- **static**：修饰变量（类变量，所有实例共享一份）、方法（不能直接访问实例成员，不能被重写只能被隐藏）、代码块（类初始化时执行一次，先于构造方法）、内部类。
- **final**：修饰**类**不能被继承（如 String）；修饰**方法**不能被重写；修饰**变量**只能赋值一次——基本类型值不变，引用类型**引用不变但对象内容可变**。

14. 深拷贝与浅拷贝、对象创建方式

- **浅拷贝**：复制对象本身，引用类型字段只复制引用，新旧对象共享内部对象；`Object.clone()` 默认浅拷贝。
- **深拷贝**：递归复制所有引用对象。实现：重写 `clone()` 逐层克隆；**序列化 + 反序列化**；手动递归复制（或 JSON 工具转换）。

- **创建对象的方式**：`new`；**反射**（`Constructor.newInstance()`，`Class.newInstance()` JDK 9 起废弃）；`clone()`（不调用构造器）；**反序列化**（不调用构造器）；工厂方法（如 `Integer.valueOf`）。

四、常用类与关键方法

15. String 不可变与字符串常量池

- **不可变原因**：类为 `final`，内部存储数组 `private final`（JDK 9 起由 `char[]` 改为 `byte[]` + coder，节省 Latin-1 字符空间），且不对外提供修改方法。
- 不可变的好处：**线程安全**、**可缓存 hashCode**（适合做 HashMap key）、支持**字符串常量池**复用、安全（参数不被篡改）。
- 常量池：字面量 `"abc"` 放入字符串常量池（JDK 7 起位于堆中）复用；`new String("abc")` 会在堆上新建对象（若池中没有 “abc” 还会在池中创建）。
- 编译期常量折叠：`"a" + "b"` 编译时合并为 `"ab"`；`intern()` 返回池中引用。

16. String、StringBuilder、StringBuffer

特性	String	StringBuilder	StringBuffer
可变性	不可变	可变	可变
线程安全	不可变，天然可共享	不安全	方法加 `synchronized`，安全
场景	少量拼接、存储传递	**单线程频繁拼接（首选）**	多线程共享同一实例

- 循环内用 `+` 拼接会反复创建对象，应改用 StringBuilder。
- StringBuffer 单个方法线程安全，**「先判断再追加」等复合操作仍需额外同步**。

17. == 与 equals、hashCode

- `==`：基本类型比较**值**；引用类型比较**是否同一对象（地址）**。
- `equals`：Object 默认实现等价于 `==`；String、Integer 等重写为**比较内容**。
- **equals 与 hashCode 约定**：
  - equals 为 true → hashCode **必须相等**；
  - hashCode 相等 → equals 不一定为 true（哈希冲突）；
  - **重写 equals 必须重写 hashCode**，否则 HashMap/HashSet 中「业务相等」的两个对象落到不同桶，导致查不到或去重失败。
- 作为 HashMap key 后**不要修改参与 hashCode 计算的字段**（建议用不可变字段）。

18. Object 类的方法

- `equals()`、`hashCode()`、`toString()`（默认 `类名@哈希码十六进制`）。
- `getClass()`：返回运行时类，`final` 不可重写。

- `clone()`：`protected`，需实现 `Cloneable`，否则抛 `CloneNotSupportedException`；默认浅拷贝。
- `wait()`（3 个重载）、`notify()`、`notifyAll()`：必须在 `synchronized` 块中调用，否则抛 `IllegalMonitorStateException`；wait 会**释放锁**。
- `finalize()`：JDK 9 起废弃、JDK 18 起标记为待移除，执行时机不确定，资源释放应使用 try-with-resources。

五、泛型、反射与注解

19. 泛型与类型擦除

- 作用：**编译期类型检查**（避免运行时 `ClassCastException`）、**消除强制转换**、**代码复用**。
- **类型擦除**：编译后类型参数替换为上界（无上界则 Object），必要处插入强转；`List<String>` 和 `List<Integer>` 运行时是同一个 Class。
  - 类、字段、方法声明上的泛型信息仍保存在 class 文件 Signature 属性中，可通过反射读取。
- 限制：不能用基本类型（`List<int>` 不合法）；不能 `new T()`；不能创建泛型数组；静态成员不能使用类的类型参数。
- **通配符**：`List<Integer>` 不是 `List<Number>` 的子类型。
  - `? extends T`：上界，适合**读取（生产者）**；
  - `? super T`：下界，适合**写入（消费者）**。即 **PECS**：Producer Extends, Consumer Super。

20. 反射

- 定义：**运行时**获取类的结构信息（构造器、方法、字段），并动态创建对象、调用方法、访问字段。
- 核心类：`Class`（获取方式：`类名.class`、`obj.getClass()`、`Class.forName("全限定名")`）、`Constructor`、`Method`（`invoke`）、`Field`。
- `getMethod` 获取 public 方法（含继承）；`getDeclaredMethod` 获取本类声明的所有方法（含 private，不含继承）；访问私有成员需 `setAccessible(true)`，受 Java 9 模块系统封装限制。
- 应用：Spring IoC 创建 Bean 与依赖注入、**动态代理**（AOP）、JDBC `Class.forName` 加载驱动、ORM 字段映射、JSON 序列化、注解解析。
- 缺点：绕过编译期检查，错误延迟到运行时；有性能开销（可缓存 Method 对象）；破坏封装。

21. 注解

- 本质：继承 `java.lang.annotation.Annotation` 的**特殊接口**；运行时通过反射获取到的是 JDK 生成的**动态代理对象**，属性值通过 `AnnotationInvocationHandler` 从 `memberValues` Map 中取得。

- **元注解**：
  - `@Retention`：`SOURCE`（仅源码，如 `@Override`）、`CLASS`（保留到 class 文件，默认）、`RUNTIME`（运行时可反射读取）；
  - `@Target`：作用位置（TYPE、FIELD、METHOD、PARAMETER、CONSTRUCTOR 等）；
  - `@Documented`、`@Inherited`、`@Repeatable`（Java 8）。
- 解析方式：编译期（注解处理器 APT，如 Lombok）或运行期反射（`getAnnotation`、`isAnnotationPresent`）。

六、异常

22. 异常体系

- 根类 `Throwable`：
  - **Error**：严重错误，程序通常无法处理，如 `OutOfMemoryError`、`StackOverflowError`；
  - **Exception**：
    - **受检异常（Checked）**：编译期必须 `try-catch` 或 `throws` 声明，如 `IOException`、`ClassNotFoundException`；
    - **非受检异常（RuntimeException 及子类）**：程序逻辑错误导致，编译器不强制处理，如 `NullPointerException`、`ArrayIndexOutOfBoundsException`、`ClassCastException`、`IllegalArgumentException`。
- 非受检异常 = RuntimeException + Error。

23. 异常处理机制

- `try-catch`：多个 catch 时**子类异常在前、父类在后**；无匹配则向调用方传播。
- `throw` 在方法体内抛出异常对象；`throws` 在方法签名上声明。
- `finally`：通常一定执行（JVM 退出 `System.exit`、进程被杀等情况除外）。
  - **finally 中的 return 会覆盖 try 中的 return**（`try{return "a"} finally{return "b"}` 返回 “b”），还会吞掉异常，**不要在 finally 中 return**；
  - 若 finally 只修改变量而不 return，try 中 return 的基本类型值已暂存，不受影响。
- **try-with-resources**（Java 7）：资源实现 `AutoCloseable` 后自动关闭，按声明**逆序**关闭；close 抛出的异常作为 suppressed 异常附加到主异常。
- 规范：不要空 catch；包装异常时保留 cause；能处理就处理，不能处理就上抛。

七、Java 新特性、序列化与 IO

24. Java 8 新特性

- **Lambda 表达式**：`(参数) -> 表达式`，简化**函数式接口**（仅一个抽象方法，`@FunctionalInterface`，如 `Runnable`、`Comparator`、`Function`、`Predicate`、`Supplier`、`Consumer`）的实现。
- **方法引用**：`System.out::println`、`String::length`。
- **Stream API**：链式处理集合，**中间操作**（`filter`、`map`、`sorted`、`distinct`，惰性求值）+ **终端操作**（`collect`、`forEach`、`count`、`reduce`）；`parallelStream()` 基于 ForkJoinPool 公共池，适合 CPU 密集且无共享状态的任务。
- **Optional**：包装可能为 null 的值，`ofNullable`、`orElse`、`map`，减少 NPE。
- 接口 default / static 方法；新日期时间 API（`LocalDateTime`，不可变、线程安全）；`CompletableFuture` 异步编排。

25. Java 17 / 21 常见新特性

- **Java 17（LTS）**：`record` 记录类（自动生成构造器、equals/hashCode/toString，Java 16 正式）、`sealed` 密封类（限制可继承的子类，Java 17 正式）、文本块 `"""`（Java 15 正式）、`switch` 表达式（Java 14 正式）。
- **Java 21（LTS）**：**虚拟线程**（JEP 444，轻量级用户态线程，适合大量阻塞 IO 任务）、**switch 模式匹配**（JEP 441）、**记录模式**（JEP 440，解构 record）、有序集合 `SequencedCollection`（JEP 431）。

26. 序列化

- 实现 `Serializable`（标记接口）或 `Externalizable`；通过 `ObjectOutputStream.writeObject` / `ObjectInputStream.readObject` 完成。
- **serialVersionUID**：版本号，反序列化时校验类版本是否一致；未显式声明时由编译器根据类结构计算，类修改后会变化导致 `InvalidClassException`，建议显式声明。
- `transient` 和 `static` 字段不参与默认序列化。
- 反序列化**不调用**类的构造器；Java 原生序列化存在安全风险和性能问题，跨语言常用 **JSON、Protobuf**、Hessian 等。
- 自定义序列化协议要点：魔数 + 版本 + 长度 + 类型的消息头；字段编号便于兼容演进；处理 TCP 粘包拆包；限制长度防恶意输入。

27. BIO、NIO、AIO

- **BIO**（`java.io`）：同步阻塞，一个连接一个线程，连接多时线程资源耗尽。
- **NIO**（Java 1.4，`java.nio`）：同步非阻塞 + **IO 多路复用**，三大组件：

    - **Channel** 通道（双向，可设为非阻塞）；
    - **Buffer** 缓冲区（position / limit / capacity，读写切换用 `flip()`）；
    - **Selector** 多路复用器：一个线程监听多个 Channel 的就绪事件（ACCEPT、READ、WRITE），Linux 底层通常基于 epoll。
    - 「同步」指就绪后仍需应用线程自己调用 read/write 搬运数据。
- **AIO**（Java 7，NIO.2）：异步非阻塞，操作完成后通过回调/Future 通知。
- **Netty**：基于 NIO 的 Reactor 网络框架，bossGroup 负责 accept，workerGroup 负责读写；ChannelPipeline + ChannelHandler 处理编解码与业务；EventLoop 中不要执行阻塞操作。

八、集合概览

28. 集合体系

- **Collection**（单列）：
    - **List**（有序、可重复）：`ArrayList`、`LinkedList`、`Vector`、`CopyOnWriteArrayList`；
    - **Set**（不可重复）：`HashSet`、`LinkedHashSet`（插入顺序）、`TreeSet`（排序）；
    - **Queue**：`ArrayDeque`、`PriorityQueue`、`LinkedList`、各种 `BlockingQueue`。
- **Map**（键值对）：`HashMap`、`LinkedHashMap`、`TreeMap`、`Hashtable`、`ConcurrentHashMap`。
- **数组 vs 集合**：数组长度固定、可存基本类型；集合长度可变、只能存引用类型、提供丰富操作。
- **Collection vs Collections**：前者是集合接口；后者是工具类（`sort`、`reverse`、`synchronizedList`、`unmodifiableList`）。

29. 线程安全的集合

- 早期同步容器：`Vector`、`Hashtable`、`Collections.synchronizedXxx()`（方法级 synchronized，粒度粗）。
- `java.util.concurrent` 并发容器：`ConcurrentHashMap`、`CopyOnWriteArrayList` / `CopyOnWriteArraySet`、`ConcurrentLinkedQueue`、`ConcurrentSkipListMap`、各种 `BlockingQueue`。

30. fail-fast 与 fail-safe

- **fail-fast**：`ArrayList`、`HashMap` 等迭代时检查 `modCount` 与 `expectedModCount`，遍历期间被结构性修改（非迭代器自身的增删）立即抛 `ConcurrentModificationException`。**只是尽力检测，不能作为线程安全保证**。
- **fail-safe / 弱一致**：并发容器（`CopyOnWriteArrayList` 遍历快照，`ConcurrentHashMap` 弱一致迭代器）遍历时不抛异常，但**不一定看到最新数据**。
- **不可修改 vs 不可变**：`Collections.unmodifiableList(list)` 只是只读视图，原集合修改后视图可见变化；`List.of()`（Java 9）是真正的不可变集合（且不允许 null 元素）。

九、List

31. ArrayList 与 LinkedList

对比	ArrayList	LinkedList
底层	**动态数组** `Object[]`	**双向链表**
随机访问	O(1)，实现 `RandomAccess`	O(n)
头部/中间插入删除	O(n)，需移动元素	定位 O(n)，修改指针 O(1)
尾部插入	均摊 O(1)	O(1)
内存	连续、紧凑，可能有预留空间；缓存友好	每个节点额外存 prev/next 指针

- 两者**都不是线程安全的**；实际开发中绝大多数场景用 ArrayList。LinkedList 还实现了 `Deque`，可作队列/栈（但通常 `ArrayDeque` 更快）。
- **ArrayList vs Vector**：Vector 方法加 synchronized、默认 2 倍扩容，已基本淘汰。

32. ArrayList 扩容机制

- 无参构造时为空数组，**首次 add 时才分配默认容量 10**（懒加载）。
- 容量不足时扩容为原来的 **1.5 倍**（`oldCapacity + (oldCapacity >> 1)`），若仍不够则直接用所需最小容量；通过 `Arrays.copyOf` 复制到新数组。
- 优化：预知元素数量时构造指定容量，或调用 `ensureCapacity`，避免多次扩容。

33. ArrayList 线程不安全的表现与解决

- 不安全原因：`elementData[size++] = e` 不是原子操作，并发下可能出现：
  - **元素覆盖 / 丢失**（两个线程拿到相同 size 写入同一位置）；
  - **数组越界**（都判断容量够，扩容前同时写入）；
  - size 与实际元素不一致、出现 null。
- 解决：`Collections.synchronizedList()`、`Vector`、**`CopyOnWriteArrayList`**（读多写少）、或外部加锁。
- **CopyOnWriteArrayList**：**写时复制**——写操作加锁（JDK 8 为 ReentrantLock，JDK 9+ 为 synchronized），复制一份新数组修改后替换 volatile 数组引用；读操作**不加锁**读旧数组。适合**读多写少**，缺点是写开销大、内存翻倍、读到的可能不是最新数据。

34. List 使用细节

- **遍历时删除**：普通 for-each 中 `list.remove()` 会触发 `ConcurrentModificationException`；应使用 **`Iterator.remove()`**、`removeIf()`（Java 8），或倒序下标 for 循环删除。
- `remove(1)`：`List<Integer>` 调用 `remove(int index)` **按下标删除**；按元素删除需 `remove(Integer.valueOf(1))`。

- **subList** 返回原列表的**视图**而非副本，修改会互相影响；原列表结构修改后再操作 subList 会抛 CME。
- **数组与 List 互转**：`Arrays.asList(arr)` 返回固定大小的视图（不支持 add/remove，且基本类型数组会被当成一个元素）；可 `new ArrayList<>(Arrays.asList(arr))`；List 转数组用 `list.toArray(new String[0])`。

十、Set

35. HashSet 去重原理

- **底层是 HashMap**：元素作为 key，value 是共享的占位对象 `PRESENT`。
- 去重依赖 key 的 **hashCode() + equals()**：先按 hash 定位桶，再用 equals 判断是否已存在。
- **自定义对象去重失败**常见原因：只重写了 equals 没重写 hashCode；或对象存入后修改了参与哈希计算的字段。

36. LinkedHashSet、TreeSet 与 Set 排序

- **LinkedHashSet**：底层 LinkedHashMap，维护**插入顺序**。
- **TreeSet**：底层 **TreeMap（红黑树）**，按自然顺序（`Comparable`）或 `Comparator` **排序**，增删查 O(log n)。
- **TreeSet 去重依据是 `compareTo` / `compare` 返回 0**，而不是 equals：若比较器只比较分数，分数相同的两个不同学生会被当作重复元素丢弃 → 比较器应补充唯一字段（如学号）。
- 对 HashSet 排序：转成 List 后 `sort`，或直接 `new TreeSet<>(set)`。

十一、Map

37. HashMap 底层结构（JDK 8）

- **数组 + 链表 + 红黑树**。数组每个位置称为桶（bucket），节点 `Node<K,V>` 含 hash、key、value、next。
- 默认初始容量 **16**，负载因子 **0.75**，阈值 threshold = 容量 × 负载因子；**容量始终为 2 的幂**；table 在**第一次 put 时才初始化**。
- **链表转红黑树**：链表长度 **≥ 8** 且数组容量 **≥ 64**；若容量 < 64 则**优先扩容**。红黑树节点数 **≤ 6** 时退化为链表（8 和 6 间留缓冲，避免频繁转换）。
- **JDK 7 vs 8**：7 为数组 + 链表、**头插法**、扩容需重新计算 hash；8 引入红黑树、**尾插法**、扩容按高位拆分。
- **key 可以为 null**：hash 值视为 0，放在下标 0 的桶；value 也可以为 null。

38. hash 计算与容量为 2 的幂

- 扰动函数：`hash = (h = key.hashCode()) ^ (h >>> 16)`，让高 16 位参与运算，减少低位相同导致的冲突。
- 下标：`index = (n - 1) & hash`，当 n 为 2 的幂时**等价于 `hash % n` 且位运算更快**。
- 2 的幂还使扩容时节点**要么留在原位置，要么移动到「原位置 + 旧容量」**，只需看 `hash & oldCap` 那一位。
- 传入非 2 的幂的初始容量时，`tableSizeFor` 会向上取整为 2 的幂（如 20 → 32）。

39. put 与 get 流程

- **put**：
  1. table 为空则 `resize()` 初始化；
  2. 计算 hash 和下标，桶为空直接放入新节点；
  3. 桶不为空：首节点 key 相同（hash 相等且 `==` 或 equals）则覆盖 value；是树节点则按红黑树插入；否则遍历链表，找到相同 key 覆盖，否则**尾插**，插入后链表长度达到 8 尝试树化；
  4. `++size > threshold` 则**扩容**。
- **get**：计算 hash 定位桶 → 检查首节点 → 红黑树查找 O(log n) 或链表遍历比较 hash 与 equals → 找不到返回 null。
- `get` 返回 null 无法区分「不存在」与「value 为 null」，应用 `containsKey` 判断。

40. 扩容机制

- 触发：元素个数 **size > threshold**（另外树化时容量 < 64 也会扩容）。
- 过程：新建 **2 倍**容量数组，遍历旧桶迁移；JDK 8 将链表按 `hash & oldCap` 拆分为**低位链（原下标）和高位链（原下标 + oldCap）**，保持原有顺序。
- 例：默认构造存 20 个元素 → 首次 put 初始化为 16（阈值 12）→ 第 13 个元素时扩容到 32（阈值 24），即**初始化 1 次 + 扩容 1 次**。
- `new HashMap<>(20)` → 实际容量 32，阈值 24，放 20 个元素不会再扩容；若想放 n 个不扩容，初始容量应设为 `n / 0.75 + 1`。
- **负载因子 0.75** 是空间利用率与哈希冲突概率的折中：太大冲突多，太小浪费空间、扩容频繁。

41. 为什么用红黑树而不是 AVL / 二叉搜索树

- 普通二叉搜索树可能退化为链表。
- **AVL 严格平衡**，查询略快，但插入删除需要更多旋转；**红黑树是弱平衡**，插入最多 2 次旋转、删除最多 3 次旋转，**增删效率更高**，整体性能更均衡。

- 链表长度达到 8 的概率在哈希分布良好时极低（源码注释中按泊松分布约为千万分之六），树化只是**应对极端冲突的兜底**。

42. HashMap 线程安全问题

- **JDK 7**：并发扩容时头插法导致链表**成环**，get 时死循环（CPU 100%）；也可能丢数据。
- **JDK 8**：尾插法解决了成环，但仍有**数据覆盖**（两个线程同时判断桶为空后写入）、`++size` 非原子导致 size 不准、扩容期间读到不完整数据等问题。
- 解决：`ConcurrentHashMap`（首选）、`Collections.synchronizedMap`、`Hashtable`。

43. 哈希冲突解决方法与 key 的选择

- 冲突解决：**链地址法**（HashMap 采用）、开放地址法（线性探测、二次探测，如 ThreadLocalMap 用线性探测）、再哈希法、建立公共溢出区。
- **String / Integer 适合做 key**：不可变（hash 不会变）、已正确重写 equals 和 hashCode、String 会缓存 hash 值。
- 自定义对象做 key：同时重写 equals 和 hashCode，且参与计算的字段不可变；equals 写错会导致相同 key 重复存储或取不到值。

44. Hashtable、LinkedHashMap、TreeMap

- **Hashtable**：方法级 `synchronized`，**key 和 value 都不允许 null**，默认容量 11、扩容为 2n+1，下标用取模；已被 ConcurrentHashMap 取代。
- **LinkedHashMap**：继承 HashMap，额外维护**双向链表**保证顺序：默认**插入顺序**；`accessOrder = true` 时为**访问顺序**。
  - **实现 LRU**：`new LinkedHashMap<>(cap, 0.75f, true)` + 重写 `removeEldestEntry` 在 size 超过容量时返回 true，自动淘汰最久未访问的头节点。
- **TreeMap**：基于**红黑树**，key 有序（自然顺序或 Comparator），操作 O(log n)；支持 `firstKey`、`floorKey`、`ceilingKey`、`subMap` 等范围操作；key 不能为 null（自然排序时）。适合需要**排序或范围查询**的场景。
- `keySet()`、`values()`、`entrySet()` 返回的是**视图**而非副本；遍历同时需要 key 和 value 时用 **`entrySet`**（或 `forEach`）效率最高。

45. ConcurrentHashMap 实现原理

- **JDK 7**：**Segment 分段锁**，Segment 继承 `ReentrantLock`（可重入），默认 16 个 Segment，即最多 16 个线程并发写；每个 Segment 内部是 HashEntry 数组 + 链表。
- **JDK 8**：取消 Segment，结构与 HashMap 相同（数组 + 链表 + 红黑树），**锁粒度细化到桶的首节点**：
  - **桶为空时用 CAS** 放入新节点（无锁）；
  - 桶非空时用 **synchronized 锁住首节点**再插入；

  - 发现首节点 hash 为 `MOVED`（ForwardingNode）说明正在扩容，当前线程**协助扩容**。
- **get 通常不加锁**：Node 的 `val` 和 `next` 用 **volatile** 修饰，table 也用 volatile + Unsafe 原子读，保证可见性。
- **为什么还要 CAS**：空桶插入是高频简单操作，CAS 乐观无锁性能更好；冲突时才用 synchronized（JDK 6 后锁优化使其性能足够好）。所以它是**乐观锁（CAS）+ 悲观锁（synchronized）结合**。

46. ConcurrentHashMap 扩容、size 与 null 限制

- **扩容**：多线程**协同迁移**，每个线程按步长（stride，最少 16 个桶）领取区间从后往前迁移；迁移完的桶放置 ForwardingNode，读操作遇到它会转到新表查找，写操作则先帮助扩容。`sizeCtl` 控制初始化与扩容状态。
- **size 统计**：类似 `LongAdder`，`baseCount` + `CounterCell[]` 数组分散计数，CAS 更新 baseCount 失败时改为更新某个 CounterCell；`size()` 求和，并发下只是**近似值**，不能作为精确判断依据。
- **不允许 null key / null value**：并发场景下 `get` 返回 null 时无法区分「不存在」还是「值为 null」，又无法像单线程那样用 containsKey 二次确认（两次调用之间可能被修改），存在**二义性**。
- **复合操作不安全**：先 `get` 判断再 `put` 不是原子的，应使用 **`putIfAbsent`**、**`computeIfAbsent`**、**`compute`**、**`merge`** 等原子方法。

47. HashMap、Hashtable、ConcurrentHashMap 对比

对比	HashMap	Hashtable	ConcurrentHashMap（JDK 8）
线程安全	否	是（方法级 synchronized）	是（CAS + 桶级 synchronized）
null	key、value 均可为 null	均不可	均不可
结构	数组 + 链表 + 红黑树	数组 + 链表	数组 + 链表 + 红黑树
性能	高	低（全表一把锁）	高并发下高
迭代器	fail-fast	fail-fast（Iterator）	弱一致

十二、Queue

48. Queue、Deque 与 PriorityQueue

- **Queue**（单端队列，FIFO）：两套方法——失败抛异常的 `add` / `remove` / `element`，与失败返回特殊值的 **`offer` / `poll` / `peek`**（推荐）。
- **Deque**（双端队列）：`offerFirst/offerLast`、`pollFirst/pollLast`，也可当栈用（`push/pop`）；**`ArrayDeque` 是首选的栈和队列实现**（比 `Stack`、`LinkedList` 更快），不允许 null。

- **PriorityQueue**：基于**数组实现的二叉堆，默认小顶堆**，可传 Comparator 改为大顶堆；入队/出队 O(log n)，取堆顶 O(1)；**用迭代器遍历不保证有序**，只有不断 `poll` 才按优先级输出；非线程安全，不允许 null。常用于 Top-K 问题。

49. BlockingQueue

- 阻塞队列：队列满时 `put` 阻塞，空时 `take` 阻塞；`offer/poll` 可带超时。内部基于 **ReentrantLock + Condition（notEmpty、notFull）** 实现生产者-消费者等待/唤醒。
- 常见实现：
  - **ArrayBlockingQueue**：数组、**有界**、一把锁；
  - **LinkedBlockingQueue**：链表、默认容量 `Integer.MAX_VALUE`（近似无界，**有 OOM 风险，应指定容量**）、**读写两把锁**，吞吐更高；
  - **SynchronousQueue**：不存储元素，每个 put 必须等待一个 take；
  - **PriorityBlockingQueue**：无界优先级队列；
  - **DelayQueue**：元素到期才能取出，用于延时任务。